

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	7
1.2 Описание выходных данных.....	7
2 МЕТОД РЕШЕНИЯ.....	8
3 ОПИСАНИЕ АЛГОРИТМОВ.....	10
3.1 Алгоритм метода getName класса Item.....	10
3.2 Алгоритм метода getDurability класса Item.....	10
3.3 Алгоритм метода repair класса Item.....	11
3.4 Алгоритм метода damage класса Item.....	11
3.5 Алгоритм конструктора класса Item.....	11
3.6 Алгоритм конструктора класса Item.....	12
3.7 Алгоритм деструктора класса Item.....	12
3.8 Алгоритм конструктора класса Item.....	13
3.9 Алгоритм функции repairItem.....	13
3.10 Алгоритм функции damageItem.....	13
3.11 Алгоритм функции createBrokenItem.....	14
3.12 Алгоритм функции main.....	15
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	16
5 КОД ПРОГРАММЫ.....	21
5.1 Файл Item.cpp.....	21
5.2 Файл Item.h.....	22
5.3 Файл main.cpp.....	22
6 ТЕСТИРОВАНИЕ.....	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	25

1 ПОСТАНОВКА ЗАДАЧИ

В компьютерной игре существует система предметов. Каждый предмет имеет имя и показатель прочности. В процессе работы программы предмет может быть отремонтирован, повреждён или создан как сломанный объект.

Одни операции должны изменять исходный предмет, другие — работать только с его копией. Также необходимо предусмотреть ситуацию, при которой сломанный предмет может отсутствовать.

Требуется разработать программную систему, содержащую один класс. Класс должен включать:

- одно закрытое свойство строкового типа — имя предмета;
- одно закрытое свойство целочисленного типа — прочность предмета;
- конструктор по умолчанию (устанавливает по-умолчанию имя предмета Unknown и прочность 0);
- конструктор с параметрами для установки имени и начальной прочности объекта;
- конструктор копии;
- деструктор;
- метод увеличения прочности на заданное значение;
- метод уменьшения прочности на заданное значение;
- метод получения имени предмета;
- метод получения прочности предмета.

Конструкторы и деструктор должны выводить сообщения в следующем формате:

- Item (Cnstr): <имя предмета>
- Item (ParamCnstr): <имя предмета>

- Item (CopyCnstr): <имя предмета>
- ~Item (Dcnstr): <имя предмета>

Требуется реализовать следующие функции:

- Функция, принимающая объект **по ссылке** и увеличивающая его прочность на 10.
- Функция, принимающая объект **по значению** и уменьшающая его прочность на 5. Внутри функции повреждения вывести строку: "Inside damageItem: <имя> durability = <прочность>";
- Функция создания сломанного предмета принимающая в качестве параметра целое значение - режим работы, возвращающая **указатель на объект**:
 - о если введён режим 0, функция должна вернуть нулевой указатель;
 - о в противном случае функция должна создать динамический объект со значениями:
 - имя: Broken
 - прочность: 0и вернуть указатель на него.

Алгоритм работы программы

1. Ввести имя предмета, начальную прочность и режим работы.
2. Создать объект с использованием конструктора с параметрами.
3. Передать объект в функцию ремонта.
4. Вывести:After repair: <имя> durability = <прочность>
5. Передать объект в функцию повреждения.
6. После возврата из функции вывести:After damage: <имя> durability = <прочность>
7. Вызвать функцию создания сломанного предмета.
 - 7.1. Если возвращен нулевой указатель, вывести:Broken pointer

is nullptr

7.2. Иначе вывести: Broken item: <имя сломанного предмета>

durability = <прочность сломанного предмета>

8. Освободить динамическую память.

9. Завершить работу программы.

1.1 Описание входных данных

Пример ввода:

Sword 50 1

1.2 Описание выходных данных

Пример вывода:

```
Item (ParamCnstr): Sword
After repair: Sword durability = 60
Item (CopyCnstr): Sword
Inside damageItem: Sword durability = 55
~Item (Dcnstr): Sword
After damage: Sword durability = 60
Item (ParamCnstr): Broken
Broken item: Broken durability = 0
~Item (Dcnstr): Broken
~Item (Dcnstr): Sword
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `item` класса `Item` предназначен для хранения основного предмета, создаваемого на основе введенных данных, и демонстрации передачи объекта по ссылке и по значению;
- объект `brokenItem` класса `Item` предназначен для демонстрации динамического создания объекта и проверки возврата нулевого указателя в зависимости от режима работы;
- функция `repairItem` для демонстрации передачи объекта по ссылке, при которой изменения, выполненные внутри функции, сохраняются в исходном объекте;
- функция `damageItem` для демонстрации передачи объекта по значению, при которой создаётся копия объекта через конструктор копии, а изменения не влияют на исходных объект;
- функция `createBrokenItem` для демонстрации возврата указателя из функции, включая возможность возврата нулевого указателя и динамического создания объекта;
- функция `main` для выполнения программы: ввода данных, создания объектов, вызова функций ремонта и повреждения, обработки сломанного предмета и освобождения динамической памяти;
- `nullptr` - явное обозначения нулевого указателя, не указывающего ни на какой объект.

Класс `Item`:

- свойства/поля:
 - поле для хранения имени предмета:
 - наименование — `name`;

- тип — строка;
 - модификатор доступа — private;
- o поле для хранения прочности предмета:
 - наименование — durability;
 - тип — цел;
 - модификатор доступа — private;
- функционал:
 - o метод getName — для получения имени предмета;
 - o метод getDurability — для получения прочности предмета;
 - o метод repair — для увеличения прочности предмета;
 - o метод damage — для уменьшения прочности предмета.

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода getName класса Item

Функционал: для получения имени предмета.

Параметры: нет.

Возвращаемое значение: строковое значение имени предмета.

Алгоритм метода представлен в таблице 1.

Таблица 1 – Алгоритм метода getName класса Item

№	Предикат	Действия	№ перехода
1		вернуть значение поля name	Ø

3.2 Алгоритм метода getDurability класса Item

Функционал: для получения прочности предмета.

Параметры: нет.

Возвращаемое значение: целочисленное значение прочности предмета.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода getDurability класса Item

№	Предикат	Действия	№ перехода
1		вернуть значение поля name	Ø

3.3 Алгоритм метода repair класса Item

Функционал: для увеличения прочности предмета.

Параметры: цел value - число, на которое нужно увеличить значение прочности предмета.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода repair класса Item

№	Предикат	Действия	№ перехода
1		увеличить durability на заданное значение value	Ø

3.4 Алгоритм метода damage класса Item

Функционал: для уменьшения прочности предмета.

Параметры: цел value - число, на которое нужно уменьшить значение прочности предмета.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода damage класса Item

№	Предикат	Действия	№ перехода
1		уменьшить durability на заданное значение value	Ø

3.5 Алгоритм конструктора класса Item

Функционал: конструктор класса Item без параметров.

Параметры: нет.

Алгоритм конструктора представлен в таблице 5.

Таблица 5 – Алгоритм конструктора класса Item

№	Предикат	Действия	№ перехода
1		иниц. строк. name со значением "Unkown"	2
2		иниц. цел durability со значением 0	3
3		вывести "Item (Cnstr): Unkown"	Ø

3.6 Алгоритм конструктора класса Item

Функционал: конструктор класса Item с двумя параметрами.

Параметры: строковый name - начальное значение имени предмета, цел durability - начальное значение прочности предмета.

Алгоритм конструктора представлен в таблице 6.

Таблица 6 – Алгоритм конструктора класса Item

№	Предикат	Действия	№ перехода
1		иниц. строк. name со значением name	2
2		иниц. цел durability со значением durability	3
3		вывести "Item (ParamCnstr): ", name	Ø

3.7 Алгоритм деструктора класса Item

Функционал: деконструктор класса Item.

Параметры: нет.

Алгоритм деструктора представлен в таблице 7.

Таблица 7 – Алгоритм деструктора класса Item

№	Предикат	Действия	№ перехода
1		вывести "~Item (Dcnstr): ", name	Ø

3.8 Алгоритм конструктора класса Item

Функционал: копирование класса Item.

Параметры: постоянный other - ссылка на объект класса Item для создания копии класса.

Алгоритм конструктора представлен в таблице 8.

Таблица 8 – Алгоритм конструктора класса Item

№	Предикат	Действия	№ перехода
1		иниц. строк. name со значением other.name	2
2		иниц. строк. durability со значением other.durability	3
3		вывести "Item (CopyCnstr): ", name	Ø

3.9 Алгоритм функции repairItem

Функционал: демонстрация передачи объекта по ссылке, при которой изменения, выполненные внутри функции, сохраняются в исходном объекте.

Параметры: ссылка на объект item класса Item для редактирования его прочности.

Возвращаемое значение: отсутствует.

Алгоритм функции представлен в таблице 9.

Таблица 9 – Алгоритм функции repairItem

№	Предикат	Действия	№ перехода
1		вызов метода item.repair с параметром 10	Ø

3.10 Алгоритм функции `damageItem`

Функционал: демонстрация передачи объекта по значению, при которой создаётся копия объекта через конструктор копии, а изменения не влияют на исходных объект.

Параметры: объект `item` класса `Item` для редактирования его прочности.

Возвращаемое значение: отсутствует.

Алгоритм функции представлен в таблице 10.

Таблица 10 – Алгоритм функции `damageItem`

№	Предикат	Действия	№ перехода
1		вызов метода <code>item.damage</code> с параметром 5	2
2		вывести "Inside <code>damageItem</code> : ", <code>item.getName()</code> , " durability = ", <code>item.getDurability()</code> , <code>endl</code>	∅

3.11 Алгоритм функции `createBrokenItem`

Функционал: демонстрация возврата указателя из функции, включая возможность возврата нулевого указателя и динамического создания объекта.

Параметры: цел `mode` - режим для работы функции по условию задачи.

Возвращаемое значение: указатель на динамически созданный объект или нулевой указатель.

Алгоритм функции представлен в таблице 11.

Таблица 11 – Алгоритм функции `createBrokenItem`

№	Предикат	Действия	№ перехода
1	<code>mode == 0?</code>	вернуть <code>nullptr</code>	∅
		динамически создать объект класса <code>Item</code> с параметрами "Broken" и 0 с помощью оператора <code>new</code> , сохранить его адрес в указатель <code>p</code>	2

№	Предикат	Действия	№ перехода
2		вернуть указатель p	Ø

3.12 Алгоритм функции main

Функционал: выполнение программы: ввод данных, создание объектов, вызов функций ремонта и повреждения, обработка сломанного предмета и освобождение динамической памяти.

Параметры: нет.

Возвращаемое значение: цел - код завершения работы программы.

Алгоритм функции представлен в таблице 12.

Таблица 12 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		объявл. строковой переменной name	2
2		объявл. цел. перемен. durability и mode	3
3		ввод значений name, durability и mode с клавиатуры	4
4		создание объекта item класса Item с парам. name, durability	5
5		вызов функции repairItem с парам. item	6
6		вывод "After repair: ", item.getName(), " durability = ", item.getDurability(), endl	7
7		вызов функции damageItem с парам. item	8
8		вывод "After damage: ", item.getName(), " durability = ", item.getDurability(), endl	9
9		иниц. указатель brokenItem с результатом вызова функции createBrokenItem с парам. mode	10
10	brokenItem == nullptr?	вывод "Broken pointer is nullptr", endl	11
		вывод "Broken item: ", brokenItem->getName(), "	11

№	Предикат	Действия	№ перехода
		durability = ", brokenItem->getDurability(), endl	
11		удалить указатель brokenItem	12
12		возврат 0	Ø

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-5.

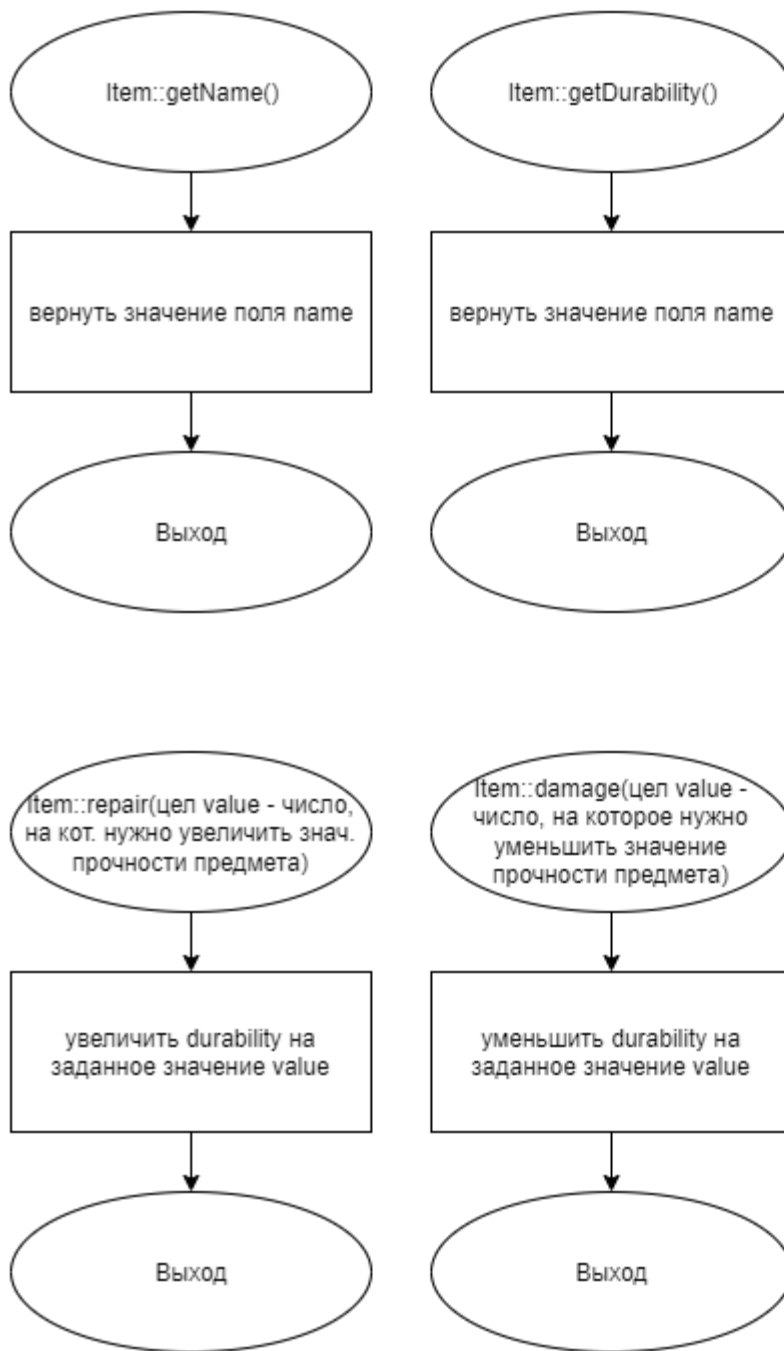


Рисунок 1 – Блок-схема алгоритма

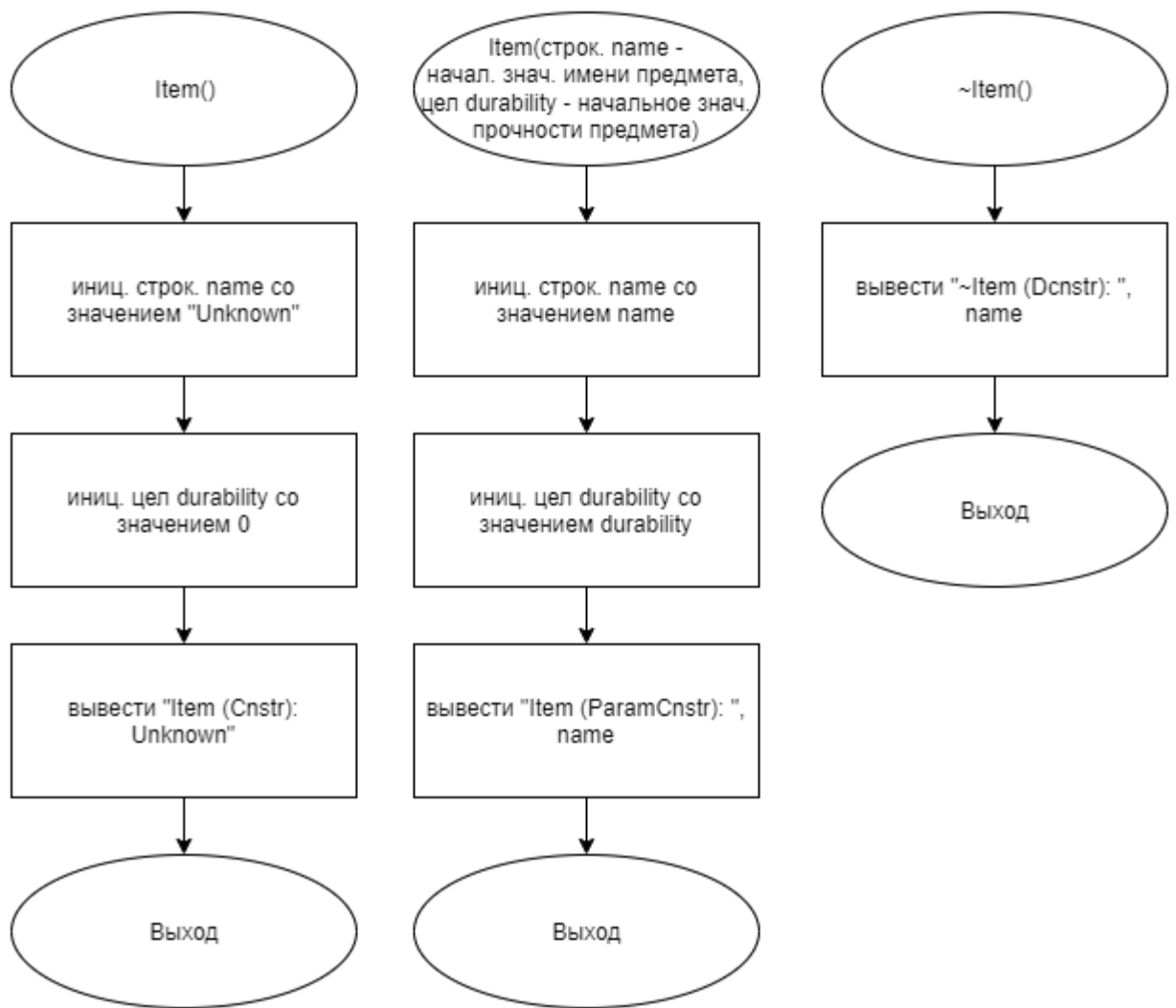


Рисунок 2 – Блок-схема алгоритма

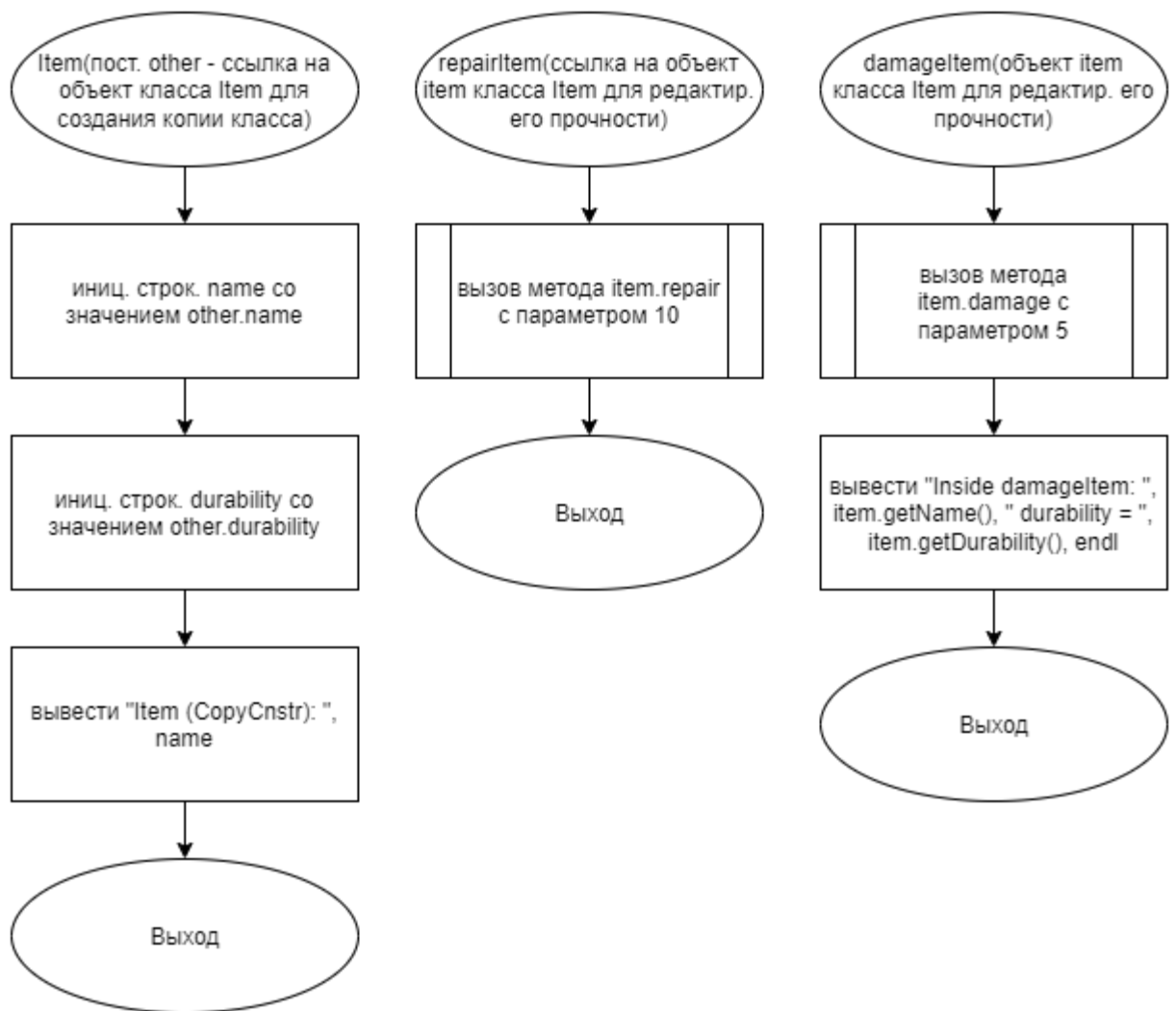


Рисунок 3 – Блок-схема алгоритма

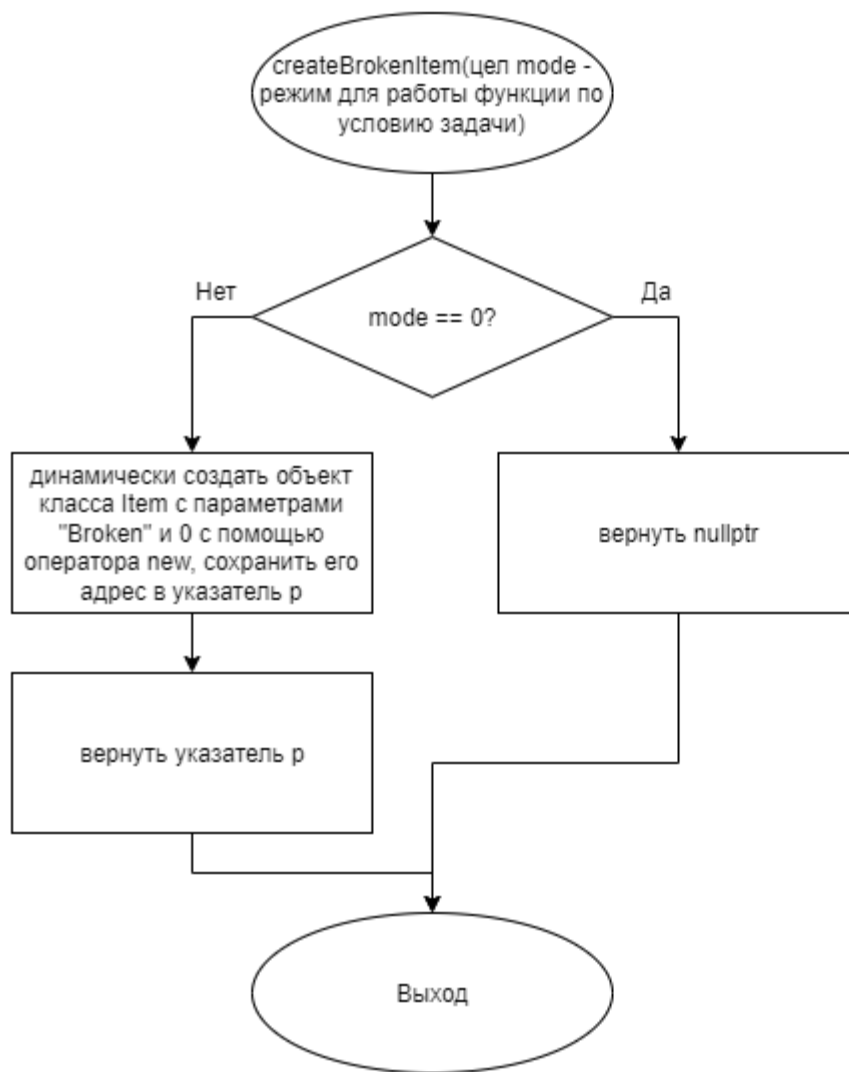


Рисунок 4 – Блок-схема алгоритма

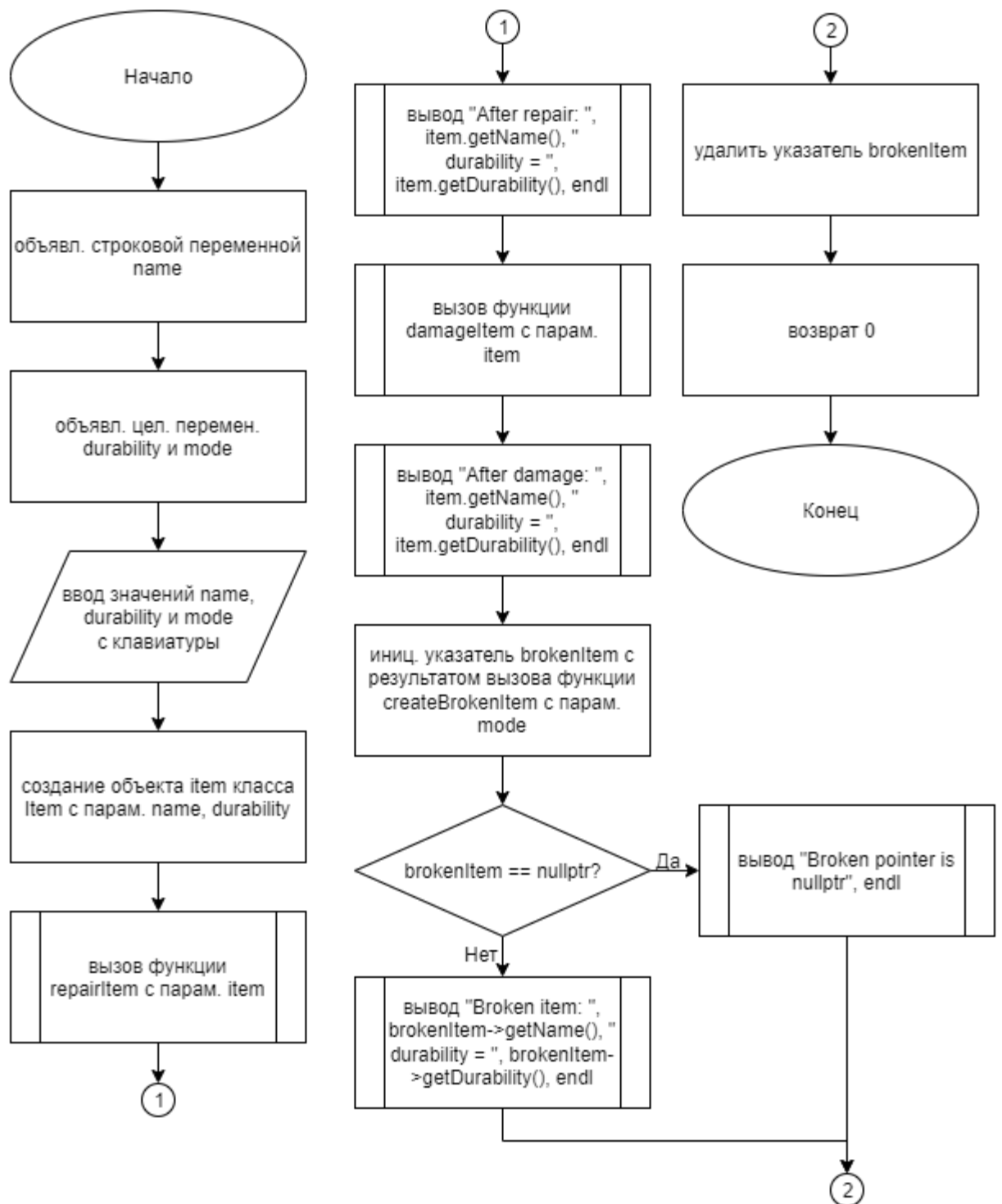


Рисунок 5 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Item.cpp

Листинг 1 – Item.cpp

```
#include "Item.h"
#include <iostream>
using namespace std;

Item::Item() : name("Unknown"), durability(0) {
    cout << "Item (Cnstr): " << name << endl;
}

Item::Item(string name, int durability)
    : name(name), durability(durability) {
    cout << "Item (ParamCnstr): " << name << endl;
}

Item::Item(const Item& other)
    : name(other.name), durability(other.durability) {
    cout << "Item (CopyCnstr): " << name << endl;
}

Item::~Item() {
    cout << "~Item (Dcnstr): " << name << endl;
}

string Item::getName() const {
    return name;
}

int Item::getDurability() const {
    return durability;
}

void Item::repair(int value) {
    durability += value;
}

void Item::damage(int value) {
    durability -= value;
}
```

5.2 Файл Item.h

Листинг 2 – Item.h

```
#ifndef __ITEM__H
#define __ITEM__H

#include <string>

class Item {
    std::string name;
    int durability;

public:
    Item();
    Item(std::string name, int durability);
    Item(const Item& other);
    ~Item();

    std::string getName() const;
    int getDurability() const;

    void repair(int value);
    void damage(int value);
};

#endif
```

5.3 Файл main.cpp

Листинг 3 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>
#include <string>
#include "Item.h"
#include <iostream>
using namespace std;

void repairItem(Item& item) {
    item.repair(10);
}

void damageItem(Item item) {
    item.damage(5);
    cout << "Inside damageItem: " << item.getName()
        << " durability = " << item.getDurability() << endl;
}
```

```

Item* createBrokenItem(int mode) {
    if (mode == 0) return nullptr;
    else {
        Item* brokenItem = new Item("Broken", 0);
        return brokenItem;
    }
}

int main()
{
    string name;
    int durability;
    int mode;

    cin >> name >> durability >> mode;
    Item item(name, durability);

    repairItem(item);
    cout << "After repair: " << item.getName()
        << " durability = " << item.getDurability() << endl;

    damageItem(item);
    cout << "After damage: " << item.getName()
        << " durability = " << item.getDurability() << endl;

    Item* brokenItem = createBrokenItem(mode);
    if (brokenItem == nullptr) {
        cout << "Broken pointer is nullptr" << endl;
    } else {
        cout << "Broken item: " << brokenItem->getName()
            << " durability = " << brokenItem->getDurability() << endl;
    }
    delete brokenItem;

    return 0;
}

```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 13.

Таблица 13 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Sword 50 1	Item (ParamCnstr): Sword After repair: Sword durability = 60 Item (CopyCnstr): Sword Inside damageItem: Sword durability = 55 ~Item (Dcnstr): Sword After damage: Sword durability = 60 Item (ParamCnstr): Broken Broken item: Broken durability = 0 ~Item (Dcnstr): Broken ~Item (Dcnstr): Sword	Item (ParamCnstr): Sword After repair: Sword durability = 60 Item (CopyCnstr): Sword Inside damageItem: Sword durability = 55 ~Item (Dcnstr): Sword After damage: Sword durability = 60 Item (ParamCnstr): Broken Broken item: Broken durability = 0 ~Item (Dcnstr): Broken ~Item (Dcnstr): Sword
Stick 1 0	Item (ParamCnstr): Stick After repair: Stick durability = 11 Item (CopyCnstr): Stick Inside damageItem: Stick durability = 6 ~Item (Dcnstr): Stick After damage: Stick durability = 11 Broken pointer is nullptr ~Item (Dcnstr): Stick	Item (ParamCnstr): Stick After repair: Stick durability = 11 Item (CopyCnstr): Stick Inside damageItem: Stick durability = 6 ~Item (Dcnstr): Stick After damage: Stick durability = 11 Broken pointer is nullptr ~Item (Dcnstr): Stick

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).